Notebook 3: Machine Learning Model

1. Problem Statement

Objective:

To build a machine learning model that classifies messages as:

- 0 → Ham (Not Spam)
- 1 → Spam

2. Import Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

3. Load Dataset

```
df = pd.read_csv('sample_data/cleaned_spam.csv')
df.head()
```

	label	message
0	0	hi how are you doing today
1	1	win money now click here
2	0	let s meet tomorrow at college
3	1	congratulations you have won a lottery claim now
4	0	can you send me the notes

Next steps: Generate code with df New interactive sheet

4. Define Features and Target

```
X = df['message']   # Feature (text)
y = df['label']     # Target (0 or 1)
```

5. Text Vectorization

Objective:

- Convert text into numerical form using Bag of Words

```
cv = CountVectorizer()
X = cv.fit_transform(X)
```

Explanation:

- Converts text into matrix of word counts
- Required for ML algorithms

6. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)
```

Explanation:

- 80% → Training
- 20% → Testing

---

7. Model Training

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

Explanation:

- Logistic Regression is used for classification problems

---

8. Model Prediction

```
y_pred = model.predict(X_test)
```

9. Model Evaluation

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.75

Classification Report:
               precision    recall  f1-score   support

           0       0.50      1.00      0.67         1
           1       1.00      0.67      0.80         3

    accuracy                           0.75         4
   macro avg       0.75      0.83      0.73         4
weighted avg       0.88      0.75      0.77         4
```

10. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[1 0]
 [1 2]]
```

Explanation:

- Shows correct and incorrect predictions
- Helps evaluate model performance

---

11. Interpretation of Results

Accuracy:

- Indicates overall correctness of the model

Precision:

- How many predicted spam are actually spam

Recall:

- How many actual spam were correctly identified

F1-Score:

- Balance between precision and recall

---

12. Machine Learning Life Cycle

- Problem Definition → Spam Detection
- Data Collection → Dataset loaded
- Data Cleaning → Done in Notebook 1
- EDA → Done in Notebook 2
- Feature Engineering → Text vectorization
- Model Training → Logistic Regression
- Testing → Test dataset
- Evaluation → Accuracy, Precision, Recall

---

13. Conclusion

- Logistic Regression successfully classifies spam messages
- Text vectorization helps convert data into usable format
- Model performance is acceptable for basic implementation
